HRMS v2.0 - SRS Requirement Status and TO DO List

Living document, keyed to the SRS. Every requirement appears three ways and all three resolve to the same row: a permanent serial (**SR-001..SR-113**) allocated in §5 document order, the SRS's own id (**FR-DOC-02**), and the requirement's sentence, which is extracted into **srs_spec.py** - the DSR layer. Verdicts come from traceability.py, which four tests keep aligned with the SRS and the live routes, so this section cannot disagree with the code. The phase history further down records what was done.

SRS requirement coverage

Requirements in the SRS (§5 + Appendix A)	113
IMPLEMENTED (code enforces it and a test covers it)	87
PARTIAL (shipped, with a named gap)	19
NOT_STARTED	6
RETIRED (superseded by v2.0)	1
Fully implemented	77%
Open, by SRS priority	High 1 · Medium 16 · Low 3 · Stretch 5
Current branch	main @ 596976a

By SRS module

Module	Done	Partial	Not started	Retired	Complete
Analytics ANL	1/3	1	1	-	■■■····· 33%
Assets AST	1/1	-	-	-	■■■■■■■■■■ 100%
Applicant tracking ATS	4/4	-	-	-	■■■■■■■■■■ 100%
Attendance & breaks ATT	15/17	1	-	1	■■■■■■■■■· 88%
Audit trail AUD	1/1	-	-	-	■■■■■■■■■■ 100%
Authentication & sessions AUTH	12/14	1	1	-	■■■■■■■■■· 86%
Documents DOC	3/3	-	-	-	■■■■■■■■■■ 100%
Expenses EXP	2/3	1	-	-	■■■■■■■··· 67%
Holidays HOL	3/3	-	-	-	■■■■■■■■■■ 100%
Scheduled jobs JOB	4/5	-	1	-	■■■■■■■■·· 80%
Leave LEA	9/10	1	-	-	■■■■■■■■■· 90%
Notifications NOT	2/3	1	-	-	■■■■■■■··· 67%
Offboarding OFF	3/3	-	-	-	■■■■■■■■■■ 100%
Onboarding ONB	6/6	-	-	-	■■■■■■■■■■ 100%
Payroll PAY	3/9	4	2	-	■■■····· 33%
Performance PERF	2/2	-	-	-	■■■■■■■■■■ 100%
Regularization REG	2/4	1	1	-	■■■■■····· 50%

Module	Done	Partial	Not started	Retired	Complete
Reports RPT	0/2	2	-	-	····· 0%
Tickets TKT	2/4	2	-	-	■■■■■····· 50%
User management USR	12/16	4	-	-	■■■■■■■■·· 75%

Development reference: open requirements

25 requirements are PARTIAL or NOT_STARTED; 1 of them are SRS **High** priority. Each row is keyed by its SRS serial (**SR-001..SR-113**), allocated in §5 document order and permanent — closing a requirement never renumbers the ones after it, so a commit, a test or a conversation can cite **SR-041** and still mean the same row next month. Per row: serial, requirement id, the SRS's own priority, the module, the routes, the **named gap** from traceability.py (not a restatement of the title), and the **next step** to open the file with.

High priority first (1). A High-priority gap outranks a Low-priority one regardless of module, which is how the SRS ranks them.

SR-011 FR-AUTH-11 HIGH NOT STARTED

Authentication & sessions · /api/mfa/enrol, /api/mfa/confirm, /api/mfa/challenge, /api/mfa/status, /api/mfa/qr, /api/mfa/disable, /api/admin/users/<emp_id>/mfa/reset

FR-AUTH-11 is therefore not implemented until MFA enforcement is deliberately re-enabled.

Next: Re-enable MFA sign-in enforcement and restore enrollment/challenge controls, then verify privileged and previously-enrolled accounts cannot establish a session with only a password.

SR-072 FR-EXP-02 Medium PARTIAL

Expenses · /api/expenses

Missing: receipt validation on upload.

Next: Validate receipts at upload through the document pipeline — extension, magic bytes and per-category size.

SR-054 FR-LEA-03 Medium PARTIAL

Leave · /api/leaves/export

The async variant for large ranges is not implemented.

Next: Run large exports through the import-job pattern: return 202 with a job id and let the worker write the file.

SR-062 FR-PAY-01 Medium PARTIAL

Payroll · /api/salary-structures, /api/payroll-runs

Missing: any surface at all for payroll RATES, which is FR-PAY-03 and has no table to read from.

Next: Grant HR read access to the payroll and salary routes (read-only, module `payroll`) and land FR-PAY-03 so there is a rates surface to read.

SR-063 FR-PAY-02 Medium PARTIAL

Payroll · /api/salary-structures

Missing: the preview endpoint, which is the half the requirement spends its second sentence on - it must call the SAME function as FR-PAY-05, "one function, not two" (Appendix A-07), and there is no preview route to be sharing it with yet.

Next: Add a preview endpoint that calls the same calculation function as run creation — the "one function, not two" rule of Appendix A-07.

SR-064 FR-PAY-03 Medium NOT STARTED

Payroll · no route yet

Every rate the calculation needs is a literal inside calc_payroll_item (app.py): PF 0.12 / ESI 0.0075 / pf_max 1800 / esi_max 21000 / pt 200 / pt_threshold 10000.

Next: Create a `payroll_rates` table (effective-dated, versioned), move the seven literals out of `calc_payroll_item`, and add an admin screen for them.

SR-065 FR-PAY-04 Medium PARTIAL

Payroll · /api/payroll-runs, /api/payroll-runs/<int:rid>/items

Three columns the requirement needs exist but are never written - tds, lop_amount and reimbursements all stay at their 0 default - so net pay silently omits TDS, omits the FR-LEA-09 loss-of-pay deduction (which would duplicate the arithmetic FR-LEA-09 exists to single-source) and omits approved unpaid reimbursements.

Next: Write `tds` from configured slabs, `lop_amount` from the FR-LEA-09 working-day figure and `reimbursements` from approved claims, then assert all three in a payroll test.

SR-066 FR-PAY-05 Medium PARTIAL

Payroll · /api/payroll-runs

Two things are missing: the duplicate (month, year, not-Cancelled) check is an app-level SELECT with no unique constraint behind it, so two concurrent requests both pass it.

Next: Add a partial unique index on (month, year) where status <> 'Cancelled', and move run creation to a background job with progress to poll.

SR-049 FR-REG-02 Medium PARTIAL

Regularization · /api/regularization

The "a specific corrected time is required" rule is not enforced: a reason-only request is accepted.

Next: Reject a reason-only regularization request with a 400 unless a corrected time is supplied.

SR-051 FR-REG-04 Medium NOT STARTED

Regularization · no route yet

No regularization Excel export.

Next: Add `/api/regularization/export` returning .xlsx, reusing the leave-export writer.

SR-106 FR-RPT-01 Medium PARTIAL

Reports · /api/reports

It ignores a supplied department only in some paths. The admin/HR view is gated by the policy.

Next: Make every admin/HR report path read the same department parameter, and add a test that the filter is applied in all of them.

SR-107 FR-RPT-02 Medium PARTIAL

Reports · /api/reports/export, /api/reports/pdf, /api/reports/department-summary

No async job for ranges beyond a month or 200 employees.

Next: Queue large exports as jobs and return 202 with a job id, as for FR-LEA-03.

SR-074 FR-TKT-01 Medium PARTIAL

Tickets · /api/tickets

The SLA target per priority is not configurable and there are no subcategories.

Next: Move the per-priority SLA targets into configuration and add subcategories with an admin picker.

SR-075 FR-TKT-02 Medium PARTIAL

Tickets · /api/tickets

Still missing: the SRS also lists "matching department" scoping, and Super Admin is not notified of a new ticket.

Next: Scope the list to a matching department for non-admin viewers and notify Super Admin when a ticket is created.

SR-016 FR-USR-02 Medium PARTIAL

User management · /api/users, /api/admin/users/<emp_id>/password

Missing: balances seeded from the grade/location policy rather than the default matrix (leave_policy derives on read instead).

Next: Seed `leave_policy_assignments` from grade and location at user creation instead of every new employee resolving to the default matrix.

SR-028 FR-USR-13 Medium PARTIAL

User management · /api/profile

The field allow-list is not a declared strict subset: an employee cannot change their own role, but the boundary is implied by the handler rather than asserted by a test.

Next: Declare an explicit field allow-list for `/api/profile` and validate against it, so the boundary is data rather than implied by the handler.

SR-029 FR-USR-14 Medium PARTIAL

User management · /api/change-password

The session token is not re-issued on change, so an existing cookie keeps working.

Next: Rotate the session id on password change and revoke the old one, so a stolen cookie dies with the old password.

SR-037 FR-ATT-07 Low PARTIAL

Attendance & breaks · /api/break-types, /api/user-breaks

The per-type summary is assembled by the client from two calls rather than served as one projection.

Next: Add a single per-type attendance summary endpoint and switch the client off the two-call assembly.

SR-013 FR-AUTH-13 Low PARTIAL

Authentication & sessions · /api/credentials

Missing the 5-minute re-authentication and the audit row for a credential read.

Next: Require a recent re-authentication for a credential read and write the audit row the SRS names.

SR-023 FR-USR-08 Low PARTIAL

User management · /api/users/<emp_id>

Missing the meta endpoint, the async CSV export and the last-50 sessions history.

Next: Add the metadata endpoint and the async CSV export, and render the last 50 sessions on the admin user detail.

SR-105 FR-ANL-02 Stretch PARTIAL

Analytics · /api/analytics/attrition-risk

The weights are hard-coded rather than configuration (FR-ANL-04), and the score is not versioned.

Next: Land FR-ANL-04 first, then store each computed score alongside the weights that produced it.

SR-113 FR-ANL-04 Stretch NOT STARTED

Analytics · /api/analytics/attrition-risk

The weights (0.4, 1.5, 0.8, 3) are literals in the handler. Changing them needs a code change and redeploy.

Next: Move the four weights into a configuration table read by the scorer, and stamp the score with the weights used.

SR-101 FR-JOB-03 Stretch NOT STARTED

Scheduled jobs · no route yet

No quarterly job opens the next performance review cycle.

Next: Create the `review_cycles` row the job opens and register the quarterly scheduler job next to the monthly accrual one.

SR-111 FR-NOT-03 Stretch PARTIAL

Notifications · /api/notification-preferences

STILL PARTIAL: no SMTP provider is configured yet, so on the current deployment nothing is actually delivered.

Next: Configure an SMTP provider and verify it with `scripts/check_smtp.py`; the outbox handler already consults the preference, so nothing else changes.

SR-070 FR-PAY-09 Stretch NOT STARTED

Payroll · no route yet

Neither half exists: there is no review_cycles table - performance_reviews rows carry a review_period string, which is a label on a review, not a cycle to report progress against - so the progress query has nothing to be progress OF, and no quarterly job opens the next cycle.

Next: Create `review_cycles`, group `performance_reviews` by cycle in the progress query, and let FR-JOB-03 open the next one.

Milestone status (delivered work by phase)

Completed tasks	41 / 48
Current branch	main @ 596976a
Next task	Phase 5: final cutover to public (traffic switch is the last operator action)
Audit pass progress	5 / 5 done
Follow-up progress	8 / 8 done
Go-live progress	7 / 13 done
Phase 0-1 progress	3 / 3 done
Phase 2 progress	2 / 2 done
Phase 3a progress	3 / 3 done
Phase 3b progress	9 / 9 done
Phase 4 progress	3 / 3 done
Phase 5 progress	0 / 1 done
Phase 6 progress	1 / 1 done

TO DO / status by phase

Phase 0-1 (3/3 done)

DONE	**Freeze the v1.0 DuckDB schema; inventory tables for ETL** Schema freeze note in docs/MIGRATION.md (08f7e40)
DONE	**Build v2.0 target schema (db/postgres_schema.sql + Alembic baseline)** 50-table public schema, CC-01..CC-16 documented (08f7e40)
DONE	**One-time DuckDB -> PostgreSQL ETL with reconciliation** hrms DB seeded; counts reconciled (08f7e40)

Phase 2 (2/2 done)

DONE	**DuckDB->psycopg adapter (db_backend.py): translate/strftime/autocommit** App runs on PostgreSQL legacy schema (b9164f2)
DONE	**Unit + browser suites green on PostgreSQL (legacy schema preserved)** 29/29 unit, 15/15 Playwright (b9164f2)

Phase 3a (3/3 done)

DONE	**Argon2id password hashing; legacy bcrypt re-hashed on login** security.py (98563ee)
DONE	**Global CSRF enforcement (fetch wrapper + csrf_token field + token API)** 98563ee
DONE	**Server-side Redis sessions (REDIS_URL opt-in) + login rate limiting** 98563ee

Phase 3b (9/9 done)

DONE	**CC-01 identity rule enforced: scripts/check_cc_rules.py + PG-gated test** 46 identity + 4 natural keys, sequences ahead of data (acb5219)
DONE	**Public-flip readiness probe (scripts/probe_public_flip.py)** GET + write-flow matrix against a throwaway public DB (acb5219)
DONE	**Boolean adapter compat: predicates, INSERT params, naive datetime round-trip** Inert on legacy (zero boolean cols); PG-gated tests (21480fe)

DONE	**Write-flow probe + salary_structures seed data fix (CC-05)** 11/11 core write flows green on public (21480fe)
DONE	**CC-09 transactional outbox (outbox.py + scheduler + admin endpoints)** Atomic business-write + event; backoff -> dead-letter (7e52f88)
DONE	**CC-07 idempotency: @idempotent decorator + idempotency_keys wired for keyed POST retries** Replay-without-duplicate, 409 on body reuse, claim released on failure; 6 unit tests green on every stack; probe replays on pure v2.0 JSONB
DONE	**Service-layer rewrite inc 1: expanded audit_log (actor/entity/entity_id/before/after/request_id, CC-13) + notifications.category (FR-NOT-03)** 8ff66bc; +6 unit tests; DuckDB 46 green
DONE	**Service-layer rewrite inc 2: shift_assignments replaces users.shift_start/shift_end (FR-ATT-17); init_db no longer mutates v2.0 public.users** get_shift/set_shift helpers reroute ~10 touch points; user CRUD + seed via set_shift; public probe 42/42 write flows
DONE	**Extend probe write section: forgot-password, payroll bank-file/TDS, ticket/ATS + verify against a clean public schema** Clean hrms_probe re-run: 94/94 GET + 42/42 write flows, including attendance, payroll, and lifecycle paths; all seed-time writes green

Phase 4 (3/3 done)

DONE	**Attendance finalisation job (FR-JOB-01)** 7 acceptance tests + nightly scheduler + regularization recompute; clean public probe 94/94 GET + 42/42 write
DONE	**Maker-checker payroll (FR-PAY-06)** Strict state machine + approval trail + Finance/Admin UI; clean public probe 94/94 GET + 42/42 write
DONE	**Corrected ATS / onboarding / offboarding flows** FR-ATS/FR-ONB/FR-OFF acceptance suite + 94/42 public probe

Phase 5 (0/1 done)

IN PROGRESS	**Final cutover: flip APP_DB_SCHEMA to public, retire legacy** Disposable rehearsal passed ETL/preflight/94-42 probe/authenticated smoke/CC-01 sequence check; maintenance-window traffic switch pending

Phase 6 (1/1 done)

DONE	**Decommission DuckDB runtime** PostgreSQL-only: APP_DB/DB_FILE, driver, legacy compose profile, ETL script and fallback lock removed; the per-session 50MB test-file leak is gone with the backend; 217 passed / 1 skipped in 67s

Follow-up (8/8 done)

DONE	**FR-USR archive/restore and session revocation** Hard delete replaced with retained archive/restore; admin UI and cross-backend tests updated
DONE	**FR-USR directory contract (bounded pagination, sorting, validation)** per_page capped at 200, allow-listed sorting, EMP/email/role/department validation, case-insensitive uniqueness, partial PUT with role-change audit, CSV import under the same contract
DONE	**FR-USR-09 permission policy (policy.py matrix + permissions API/UI)** 27-module role matrix, deny-beats-allow overrides, full-replace PUT with audit diff, anti-lockout guard, admin permissions modal
DONE	**FR-USR-15 policy enforcement wiring (decorators + navbar)** Role gates narrowed by policy.can(); navbar derived from the same gate/module tags; department grant modelled; nav/API drift tests
DONE	**FR-USR-15 inline scope checks + audited pii_reveal** No handler branches on the session role copy; can_view_all is the CC-11 scope half; PII reveal route audited per cross-employee read
DONE	**FR-LEA-06/08 policy-derived leave balances** Entitlement derived from the effective leave_policy_assignments; reserved ledger enforced; audited policy API + admin modal

DONE	**FR-USR-04 background bulk import jobs** 202 + queued import_jobs, single-claim dispatcher, progress/history/cancel, upload deleted, audited completion
DONE	**FR-USR two-person anonymisation** proposed -> confirmed by a different approver -> applied; archived-only; dry run; value-scrubbed audit history; required salt; trade-offs recorded in docs/ANONYMISATION.md

Audit pass (5/5 done)

DONE	**FR-AUTH-11 TOTP MFA (two-phase enrolment, parked half-session refused by construction)** mfa.py + 7 routes; admin reset is audited and answers identically either way; 14 unit tests + the browser suite's real MFA panel (8fbe0-series)
DONE	**FR-AUTH-03 account lockout + FR-AUTH-02 uniform 401s** lockout.py + Alembic 0009; all six refusal paths byte-identical; streak resets on success and on apply; admin unlock is NOT a status change (423bfb5)
DONE	**FR-AUTH-08/09 password reset: enumeration oracle closed, digest-only lookup** Uniform 202 with no token, delivery via outbox, sibling tokens invalidated, reset page added; the previous 200-with-token was recorded as IMPLEMENTED and was not (6fdd9ed)
DONE	**FR-AUD-01 audit ratchet over every mutating handler** AST sweep with one-level delegation resolution; subset-AND-no-stale assertion; 20 handlers -> 1 documented exemption (2b8e7e0, a754e9d, 999ff75, c464415)
DONE	**FR-EXP-03 expense state machine, FR-PERF-01/02, FR-TKT-03/04, FR-LEA-05, FR-HOL-01/02/03, FR-NOT-03** Each found by reading the handler for a neighbouring requirement; gate + state-machine + audit gaps closed; matrix 50 -> 58 IMPLEMENTED

Go-live (7/13 done)

DONE	**1. Honest email transport: send_email cannot report a send that never happened** Unconfigured SMTP is now a FAILURE, so the event retries and dead-letters; /api/health reports degraded; three tests including end-to-end dead-lettering (0e064d4)
DONE	**2. Security response headers (SRS 11.3) - none of the five shipped** Flask-Talisman; CSP is a module constant the test asserts against; no unsafe-inline for scripts; disabled outside production on purpose (0e064d4)
DONE	**3. FR-JOB-05 scheduler leader election** Redis lease, token-fenced Lua renewal, lost lease shuts the scheduler down, unreachable Redis refuses to start; SRS chaos test with 3 real processes (0e064d4)
DONE	**4. Backup + a restore drill that can fail** scripts/backup.py verify restores to a scratch DB and runs the app's own gates; row-count check; verified green AND red (dbd669c)
DONE	**5. Load test + the blocker it found: rate limits could not meet the SRS NFRs** k6 scenario + measurable harness; FR-AUTH-01 per-account keying; 175 of 375 requests were being locked out behind shared NAT (2ed778e)
DONE	**SMTP configuration that actually works (port 465, open relays, hung-relay timeout)** Transport selected by port, login skipped without credentials, SMTP_TIMEOUT_SECONDS; scripts/check_smtp.py verifies a real provider (d39b3b3)
DONE	**Admin-set password: the recovery path that needs no mail server** Sessions closed, lockout cleared, self-target needs current password, 409 names the right button; probe flow proves it on v2.0 (8f9b90b)
PENDING	**Configure real SMTP credentials and confirm a reset email is delivered** OPERATOR. Until SMTP_HOST is set, /api/forgot-password cannot deliver anything. scripts/check_smtp.py --to then /api/health must show email_configured: true
PENDING	**Install ops/cron.example (daily backup + quarterly restore drill)** OPERATOR. A backup that has never run is indistinguishable from a backup that does not exist
PENDING	**Run ops/load/load.js against staging (sustained 150 req/s + 1,000-login burst)** OPERATOR. Needs a dedicated account whose role is NOT in mfa.MANDATORY_ROLES, or the scenario refuses to run
PENDING	**WAL archiving / point-in-time recovery** OPERATOR DECISION, deliberately not implemented: PITR needs a WAL destination and retention policy belonging to whoever owns the storage. Base backups and the restore drill do not cover RPO <= 15 min

IN PROGRESS	**Delta sync -> maintenance window -> DNS/load-balancer traffic switch** OPERATOR. Only a disposable rehearsal (hrms_cutover_rehearsal) has run. Phase 5 stays IN PROGRESS until the switch is verified in production
PENDING	**K6 against staging before release + Schemathesis contract tests** OPERATOR/TOOLING. The SRS names both; the k6 scenario is written and the local harness measures the same targets, but no staging environment exists yet

Test & readiness gates

Gate	Stack	Current result
Unit suite (tests/test_app.py)	PostgreSQL legacy	351 passed, 2 skipped; includes expense-category CRUD, audit/reference guards, ClamAV protocol/fail-closed and category-cap checks, plus session expiry
Unit suite (tests/test_app.py)	PostgreSQL + Redis	Full unit suite not rerun with REDIS_URL; dedicated session-store suite reverified after the transaction retrofit
Redis session store (tests/test_redis_sessions.py)	PostgreSQL + Redis	11 passed, including absolute-expiry Redis-key revocation; all skip cleanly with REDIS_URL unset
Browser suite (tests/test_playwright.py)	temporary PostgreSQL 17, threaded server	23 passed; includes the shared pre-boarding document upload flow
CC-01 rule checker (scripts/check_cc_rules.py)	temporary PostgreSQL 17 public schema	OK - 51 identity + 5 natural primary keys; sequences ahead of data
Cutover preflight (scripts/cutover_preflight.py)	temporary PostgreSQL 17 public schema	Expected head 0015_expense_category_names; verifies identity/sequence health and the case-insensitive expense-category index
SRS extraction (scripts/extract_srs_spec.py --check)	HRMS_SRS_v2.0.pdf	113 requirements, no duplicates, srs_spec.py byte-identical to the PDF
Public-flip probe (scripts/probe_public_flip.py)	disposable PostgreSQL 17 public schema	115/115 GET served; 66/66 write flows OK, including expense-category CRUD + claim reference guard, audit/outbox, dashboard cache, break-type configuration and start-break(Tea). Scanner and S3 transports are isolated from this schema probe.
Backup restore drill (scripts/backup.py verify)	scratch DB	PASS on a real dump (exit 0); FAIL on a deliberately empty one (exit 1)
Load harness (scripts/loadcheck.py)	local server, 80 req/s	p95 181 ms, p99 226 ms, 0.000% errors, 0 lockouts - SRS targets p95<300, p99<800, errors<0.1%
Shared-NAT check (scripts/shared_nat_check.py)	15 employees, one egress IP	375/375 served after the fix; 200/375 and 175x429 before it
CI PostgreSQL job	postgres:17 + redis services	Lint, compose config, alembic upgrade head, unit (with and without Redis), browser, and the preflight/CC-01/probe gates on a clean v2.0 target

Update log (newest first)

2026-10-12	FR-AUTH-11 (High): two-factor authentication has been disabled system-wide as requested. Login now completes after password verification for every role, including accounts with an enabled saved factor. Enrollment, QR, confirmation and challenge routes return 410; the profile identifies MFA as disabled and allows removal of an old stored credential. Existing encrypted credential rows are retained rather than deleted. Traceability is corrected to NOT_STARTED because the SRS control does not protect sign-in while this switch is off.
2026-10-12	FR-EXP-01 (Medium): Admin and Super Admin can now create, partially update and delete expense categories from the expense page and API. Names are bounded, trimmed and unique across case/whitespace variants with a database index; mutations are audited, and categories referenced by claims return 409 rather than breaking retained claim history. The existing `cat_id` identity and claim foreign keys are unchanged. Alembic `0015_expense_category_names` and the compatibility boot index provide the same invariant. Unit suite passed 351 tests with 2 skips; browser suite passed 23/23. The disposable public-schema probe passed twice at 115/115 GET and 66/66 writes; CC-01 and preflight passed at head 0015.

2026-10-12	FR-DOC-02 / FR-USR-12 / FR-ONB-04 (Medium/Low): every employee and pre-boarding document upload now shares content-signature and declared-MIME checks, per-category caps, and a real ClamAV INSTREAM scan. Caps are 5 MiB for payslips, salary slips, bank details and offer letters; 10 MiB for identity/address proofs; 15 MiB for education and certification; 20 MiB for Other and unlisted categories. The existing 50 MiB HTTP ceiling remains. A missing, timed-out or erroring scanner refuses the upload; the production Compose stack now includes a private ClamAV service and waits for its health check. Pre-boarding files were still local despite the previous matrix claim, so they now use the same private object store and presigned download path. Main document keys are opaque 192-bit random values rather than employee ids, sequence ids and filenames. Tests exercise the INSTREAM framing, clean/infected/unavailable outcomes, all category caps, upload refusal, random-key persistence and download lookup. Unit suite: 350 passed / 2 skipped; browser suite: 23/23; lint, compile and both Compose configs passed.
2026-10-12	FR-AUTH-04 (Medium): authenticated sessions now have an immutable 24-hour absolute deadline, checked before protected requests; expiry closes the attendance row, writes `SESSION_EXPIRED`, clears the identity and revokes Redis storage. Missing timestamps on old sessions fail closed. Implementing the absolute limit exposed that the signed-cookie path never set Flask's permanent-session flag, so its claimed 8-hour idle expiry was not actually enforced. Login now enables rolling permanent sessions, giving both signed cookies and Redis the 8-hour activity-refreshed limit independently of the fixed deadline. Tests cover idle cookie max-age, an unchanged absolute timestamp across activity, attendance close and audit; the Redis suite also proves key revocation. Full unit validation passed 346 tests with 2 skips; all 11 Redis session tests passed. Browser tests passed 23/23. On a fresh public schema, the probe served 115/115 GET routes and 64/65 writes, with only the existing guarded `leaves(apply)` 409; CC-01 and the read-only preflight passed.
2026-10-12	FR-AUD-01 (Medium): audit writes now travel through an `audit.recorded` outbox event and materialize inside the same request transaction as the business write. Mutating requests share one transaction; `get_db()` connections and nested CC-09 transactions use savepoints, preserving legacy close/commit/rollback call patterns. Error responses roll business changes back while preserving the audit of an attempted action; login 401s commit lockout counters. Hourly maintenance and import completion now bind their audited state transitions too. Delivered audit-event payloads are cleared after materialization. The last unaudited route, marking one's own notifications read, now emits one aggregate audit row; the AST ratchet has an empty unaudited set. The atomicity/outbox tests and full unit suite passed 343 tests with 2 skips. The public probe verified the canonical audit row and delivered outbox event; total was 115/115 GET and 64/65 writes, with the existing `leaves(apply)` 409 guard. Browser validation had one known CDN navigation timeout (22 passed); the failed test passed in isolation.
2026-10-12	FR-ATT-15 (Medium): `/api/dashboard-stats` now serves the stable SRS keys (`total`, `logged_in_today`, `on_break`, `blocked`, `pending_leaves`) from a Redis cache with a 15-second TTL. A scheduler worker refreshes every 10 seconds, leaving room for the TTL and avoiding per-request recomputation; a cache miss warms once under a lock. Existing `total_employees` and `blocked_users` aliases remain so the current admin dashboard keeps rendering unchanged. With no Redis configured the same TTL is provided in process memory, matching the project's optional-Redis dev mode. Cache, fixed-TTL fallback, payload and scheduler tests were added. The focused dashboard/break/traceability set passed 27/27 and the full unit suite passed 341 with 2 skips. On the public schema, the cache worker flow passed; the probe reported 64/65 writes with its existing `leaves(apply)` 409 guard and no unguarded failures.
2026-10-12	FR-ATT-02 (Medium): `POST /api/start-break` now runs the quota/approval checks, auto-end and new Active row in one PostgreSQL transaction; a failed insert rolls the old break's end back as well. The compatibility schema now creates the same `uq_one_active_break` partial unique index already present on the canonical schema, enforcing at most one Active break per employee under concurrency. The compatibility suite had two fixtures that deliberately stored two simultaneous Active rows for one employee; with the rule now real, the orphan-sweep fixture uses two employees and a notification assertion is scoped to its subject. Regression tests prove rollback and database-level duplicate refusal. Break/policy/traceability checks passed 22/22; after correcting the neighboring delegation test's month-boundary fixture, the full unit run passed 341 with 2 skips. The public-flip probe served 115/115 GETs and 64/65 writes, with the existing `start-break(Tea)` flow passing and the unrelated `leaves(apply)` flow guarded with 409.

2026-10-12	FR-ATT-01 (Medium): break types are now managed through admin-gated create/update/delete routes. The existing `break_type` key and foreign keys remain intact; names are additionally case-insensitively unique, so a type can have one optional location scope but cannot have independently configured duplicates at several locations. The location is resolved from the employee's effective leave-policy assignment, and location-scoped types are filtered from both the list and write APIs. `requires_approval` is explicit, Lunch defaults to true, and a pending request cannot start the break until it has been approved. Alembic `0014_break_type_config` and the compatibility schema add matching columns. Unit regression tests cover CRUD, role denial, location visibility, approval flow and retained references; the public-schema probe created, updated, read back and deleted a configured type. The focused break/policy checks passed 21/21; CC-01 and preflight passed at head 0014. The full public probe served 115/115 GETs and 63/64 write flows; its unrelated `leaves(apply)` flow returned a guarded 409.
2026-10-12	FR-ANL-01 (Medium): all four analytics aggregates are now materialised in `analytics_snapshots`, refreshed by a 30-minute scheduler job and available through an admin-gated refresh route. Read paths serve a fresh snapshot with an `as_of` timestamp; missing or stale data is recomputed live and stored as a warm-up fallback. Leave trends retain their request window by storing a wider monthly series and filtering it at read time. The SRS says materialized views; this implementation uses a keyed JSON snapshot table because the trend window is parameterised and the app supports two schema shapes. The metric is its natural primary key and is recorded in the CC-01 whitelist; the public schema preflight requires the new table. Targeted analytics, traceability and CC-01 tests passed (9/9); public-schema JSONB round-trip and the authenticated refresh route passed. The updated page accepts both the old array and new timestamped response shapes. The full unit run had 335 passed / 2 skipped and one unrelated existing failure: the regularization-delegation test creates a date 21 days ahead but filters by the current month, so it fails when the date crosses a month boundary. Browser suite passed 23/23, including the updated analytics page. The requirement is now IMPLEMENTED; the next listed item is FR-ATT-01.
2026-10-09	FR-PAY-07 / FR-DOC-02 / FR-DOC-03 / FR-ONB-04 (High/Medium): object storage (S3/MinIO) and presigned URLs for payslips and documents. A new `object_storage.py` module provides a boto3-based abstraction over S3-compatible backends (MinIO for dev/CI/tests, AWS S3 for production). Payslips (FR-PAY-07): the PDF is generated, uploaded to object storage, and served via a presigned URL -- the route returns a 302 redirect. Documents (FR-DOC-02): uploads validate content (magic bytes, EICAR), upload to object storage under a random key, downloads return a 302 to a presigned URL, and deletes remove the object from storage. FR-DOC-03 (document downloads) now uses the same presigned URL mechanism. FR-ONB-04 (pre-boarding documents) inherits the completed document pipeline. Moto mocks S3 in unit/browser tests (no MinIO container needed). The earlier claim that 'a renamed .exe passes' was incorrect - the content is validated against the claimed extension magic numbers and EICAR is rejected. A later FR-DOC-02 slice completed the per-category size policy, integrated fail-closed ClamAV scanning and moved pre-boarding documents into the shared object-storage path.
2026-10-08	FR-ATT-05 (High): 'one Pending Lunch approval per employee per shift date' is now enforced by a unique **partial** index on both schemas, not just an app check. The canonical schema already had `uq_pending_lunch_approval`; `init_db` now creates the same index on the compatibility shape (PostgreSQL is the only backend since the Phase-6 DuckDB decommission, so the partial-index limitation that forced conditional INSERTs elsewhere no longer applies), and the route translates a concurrency UniqueViolation race into the same 409 instead of a 500. The index is partial by design - an Approved/Rejected row frees the slot for a fresh request - and three unit tests assert the sequential rule, the DB-level concurrency refusal (twin direct inserts, one winner), and the index metadata on the connected schema; the probe gained a write flow that bypasses the route on purpose to prove the canonical index refuses the duplicate. Verifying the browser suite also surfaced a pre-existing FR-JOB-05 wiring defect: the lease-renewal job was installed unconditionally, and on the no-Redis fallback `renew()` correctly returns False, so the dev/CI scheduler shut itself down ~20 seconds after start - visible only as a stray 'Scheduler lease lost' log line in an unrelated test's captured output. Renewal is now installed only under `scheduler_leader.renewal_required()`, with a truth-table test and an AST wiring test.

2026-10-08	FR-LEA-08a at last: the approval-delegation slice, the last NOT_STARTED row in the leaves module. The table existed in the canonical schema with a no-overlap exclusion and no route anywhere. `delegations.py` owns one rule in two shapes - `can_approve_for` for a single request, `approvable_employees` for the pending_my_approval lists - because two implementations of 'who may decide' would drift the first time a third approval path appears. The rule: a delegation is active when today falls in its date range, an overlap is refused by the app and by the GiST exclusion on the canonical schema, creating one requires actually holding authority (the delegator must manage somebody or be an admin), self-delegation is a 409, an employee cannot file one in someone else's name (403 - the CC-10 half that lives in this feature), and only the delegator or an admin may revoke, so a delegation left behind by someone who has since left can still be cleared. All six approve/reject routes (leaves, regularization, breaks) now share one _approval_denial and fire `approval_note` - 'approved by delegate for manager X', the SRS's own wording - only when the delegation is what authorised the decision, with the notification landing in the new `Approvals` category. The gate fix is the fifth instance of the same class: `reporting_line_required` now also admits an active delegate who manages nobody, because the delegate case exists precisely for a plain employee standing in for their manager. Two matrix rows were corrected with it: FR-LEA-04's note asserted 'not the applicant' while nothing actually refused the wrong approver - `can_approve_for` now refuses actor == target first, whatever their role - and FR-ATT-06's open gap, 'Delegated approvers still not consulted', is what this slice closes. FR-REG-01's manager view includes delegated reports because the pending_my_approval filter is one shared clause across both lists; the regularization approve/reject gate widening from admin-only to reporting_line_required is recorded as deliberate, since a manager view an only -admin can decide is a dead end. Matrix 74 IMPLEMENTED / 33 PARTIAL / 5 NOT_STARTED / 1 RETIRED. Unit 328 passed / 2 skipped, browser 23/23, v2.0 gates probe 115/115 GET + 60/60 write (run twice for idempotency), CC-01 and the read-only preflight green at head 0012_approval_delegations_compat.

2026-10-07	Asked to key the to-do list to the SRS and the DSR (Detailed Software Requirement) and make it usable as a development reference with allocated serial numbers. The request exposed that the document was keyed to neither. Three findings, in the order they mattered. FIRST: the ID set the matrix was tested against was a hand-copied literal in tests/test_app.py, extracted with a plain-text regex over the whole PDF - and plain-text extraction renders the nine payroll ids as `FR-PAY -01` (the PDF's justification inserts a space before the hyphen), so FR-PAY-01..09 were not tracked at all: nine requirements of which four are High or Medium priority had never appeared on a to-do list. SECOND: the matrix's own priority/delta columns had no source to be checked against, and had drifted on 43 of 104 rows - seven where the SRS says High and the matrix said Medium (so the list's 'High priority first' ordering left them out of the High group), seventeen the other way round. THIRD, and the reason the document could not be used as a reference: it answered 'what did we do', never 'what does the SRS actually ask for'. So the DSR is now a layer of its own - `srs_spec.py`, generated from §5 by scripts/extract_srs_spec.py, holding each requirement's id, its own sentence, the Pri and Δ columns, its provenance and a permanent serial. The extractor parses the PDF in LAYOUT mode rather than plain text, because the table is a four-column geometry and a row begins at column 0 - which recovers the nine FR-PAY ids that keyword matching could not. 113 rows come out (112 from §5 plus FR-ANL-04, which is printed only in Appendix A and is serialised last so it cannot renumber anything). Serials are allocated in SRS document order, not matrix order, which is the property that makes `SR-041` citable in a commit and still meaningful next month: closing a requirement never renumbers the ones after it. The matrix now covers all 113 (70 IMPLEMENTED / 36 PARTIAL / 6 NOT_STARTED / 1 RETIRED), its priority/delta drift is corrected from §5 and held there by a test, and `test_srs_spec_matches_the_srs_pdf` re-extracts the PDF and fails if the committed copy disagrees - a committed extraction that no longer matches the spec reads as authority and is worse than no extraction. Getting the statements *right* took one more fix, and it was not a parser bug: layout mode renders a gap as round(gap / space_width) per font, and the SRS's bold face - which sets the first line of most §5 statements - declares a space of 838 units while the document actually emits word gaps of 248-394, so every gap rounded to zero and 37 statements read `Approvaldelegation.`, `Theworking-day/`, `Dailyattendance`. The word space is measurable from the document itself (the most frequent TJ adjustment, since word gaps repeat and letter kerns do not), so the extractor measures it per face and corrects space_width only where the declared value would round the document's own gap to zero - that condition *is* the defect, so of the SRS's five faces only the bold one is touched. 37 of 113 statements changed and not one gained a double space. The `AppendixA` repair was deleted rather than left as a no-op: a repair that hides the defect the extractor no longer has is how a regression stays invisible. The list itself gained a DEVELOPMENT REFERENCE section: one row per open requirement carrying serial, SRS id, priority, module, routes, the named gap and a hand-written NEXT STEP, which is what separates a reference from a status report. The next steps are deliberately not derived - the gap extractor can say what is missing, only somebody who has read the handler knows which file to open first - and `_NEXT_STEPS` is held against the open set by a test, so a requirement that turns PARTIAL fails the build until someone decides what to do about it, the same ratchet the unaudited-handler list uses. Two extraction bugs were fixed while writing it: the sentence split consumed the period, so gaps came out as run-on prose ("...are literals in the handler Changing them needs a code change and redeploy"), and promoting a note's `; ` to `. ` left the following clause lower-case. Reportlab's paraparser also eats `` as an XML tag, so the escaping moved into `_escape_markup` and now covers every dynamic string in the section rather than just the routes - the module names have carried a bare `&` the whole time. Matrix 70/36/6/1, srs_spec 113 rows, unit suite green.

2026-10-02	Asked to check whether the breaks module was complete: 9 of 17 FR-ATT rows were IMPLEMENTED, 7 PARTIAL, 1 retired - so no, and the ToDo PDF listed all seven. Started with FR-ATT-09 (High), which turned out to be the second instance of the FR-LEA-09 defect class. The SRS asks for shift_hours = last_logout - first_login (NOT the sum of sessions) when both exist, else now - first_login for an open shift, CAPPED at the scheduled shift length +25% and flagged estimated: true. The cap and the flag were both missing - and the miss was that _attendance_worked_hours already implemented the cap while /api/user/shift-summary did not. So an employee who forgot to log out saw a figure that grew without limit on their own dashboard while payroll was credited the capped one: a support call every time, and an employee whose screen disagrees with their payslip has no reason to trust either. shift_hours.py is now the single rule called from both, with a test walking the AST of app.py so a third implementation cannot appear. The cap is on the OPEN-SHIFT branch only - the SRS attaches it there, and a closed shift is real recorded data, so capping it would under-credit genuine overtime, which is a payroll decision rather than a data-quality one; the payroll credit ceiling stays on top for the same reason. Four judgement calls recorded in the matrix: estimated is reported for any open shift (a superset of the SRS wording, serving its intent); capped says separately whether the allowance actually bound; scheduled_hours and shift_configured are reported because an employee with NO configured shift resolves to midnight-to-midnight = a 24h span, so the +25% cap would be 30 hours and the skew this requirement prevents would arrive by another route - the module's documented 8h default applies instead and shift_configured says so; and a night shift whose end precedes its start has its negative span rolled forward a day rather than clamped to zero, which would otherwise make every overnight shift look instantaneous and cap it at zero hours - an employee on nights credited nothing. Two of my own bugs, both found by tests written to be time-independent: a textual check for the old inline 'now - first_login' matched this file's own COMMENT describing the defect it removed, so an AST check replaced it (a source-text search in a codebase that documents its own bugs will always find them in the prose); and a literal '== 10.0' assertion made the route test depend on the wall clock, since the session was dated today and early in the morning the elapsed figure sat under the cap - now it asserts the invariant the rule guarantees rather than a number. One unreproduced single flake, recorded rather than hidden: test_defaults_are_true_and_a_notification_is_delivered_by_default failed once in a full run and passed in 2 subsequent full runs and 3 in isolation, so it is not evidence of a regression and was not chased further. Matrix 68 IMPLEMENTED / 31 PARTIAL / 4 NOT_STARTED / 1 RETIRED. Unit 314 passed / 2 skipped, browser 23/23, v2.0 gates 113/113 GET + 59/59 write, probe run twice.
2026-10-02	FR-USR-07: bulk block/unblock/archive/restore. High priority, retained from v1.0, and entirely unimplemented - the single routes took one employee at a time, so offboarding a department was one request per person and an administrator interrupted halfway had no way to tell which half. All three SRS clauses ship, and the DELEGATION is the design rather than an implementation detail: every row calls _set_user_access_status, the same function the single routes call, so the batch has no behaviour of its own to get wrong. It cannot be more permissive about a 409, cannot forget to close sessions, and cannot skip the audit row, because it has no implementation of its own. Every refusal the single routes make - already archived, archived users must be restored first, already blocked - is inherited per row with the same wording, and a test asserts that inheritance directly by archiving twice and then blocking an archived user. Self is a per-row FAILURE rather than a silent skip, carrying the single routes exact 409 message: silently dropping the row would leave an administrator who selected thirty people seeing '29 archived' and no reason the thirtieth was different. Status codes state which outcome happened - 200 all, 207 mixed, 400 none - because a 200 that hid a failure or a 400 that hid twenty successes is the reason the SRS asks for per-row results at all. The action vocabulary is data (BULK_USER_ACTIONS) mapping each action to its status and allow_login value, because those two decide whether sessions are closed, and a test proves a bulk block closes one session per employee. Validation runs before anything is touched, ids are normalised and blank-filtered up front so [''] gets the same answer as [], and the batch is bounded at 500 so one request cannot hold locks across the directory. The admin UI gains a selection column and a bulk bar whose result renders the per-row list rather than a single done, with select-all scoped to the current page and saying so in its title. No schema change and no Alembic revision: the operations already existed and only the entry point was missing. Debugging the probe flow for this one cost four wrong-step bugs, all mine, and the reason is recorded because it is a class: an inverted assertion (if not results[x].get('ok') is true precisely when the row correctly failed, so a WORKING route was reported broken - an inverted probe assertion is worse than a missing one because it looks like evidence); fixture ids with letters that the EMP\d{3,} directory contract rejected at the create step, so the flow returned that 400 and pointed at the bulk route; probe fixtures cleaned up only on the happy path, so three failing runs left three sets of archived employees behind; and every failing check returning a bare 409, which made 'the guard fired' indistinguishable from 'the wrong step ran'. run() now accepts a (status, detail) tuple and every check in this flow names itself. Matrix 67 IMPLEMENTED / 32 PARTIAL / 4 NOT_STARTED / 1 RETIRED. Unit 310 passed / 2 skipped, browser 23/23, v2.0 gates 113/113 GET + 59/59 write, probe run twice with zero leaked fixtures.

2026-10-02	FR-LEA-07: HR/Admin can grant leave days by hand. High priority, and nothing implemented any of it - an administrator who needed to give someone three days had no route at all and would have gone to a database console. All five SRS clauses ship: add days to one or more employees for a type/month/year, audited as LEAVE_GRANT with before/after totals, the employee notified, and a GET returning the grant history. THE DESIGN DECISION carries the slice: a grant is NOT written to leave_balance.total_days. That column is DERIVED - ensure_balances recomputes it from the policy on every read and overwrites it - so a grant written there would be silently erased the next time anybody opened the balance, surviving only until the next page load with no audit row able to explain where it went. The grant is a ROW in leave_grants (Alembic 0011) and entitlement_days adds the year's grants to the policy figure. The entitlement was split into a wrapper plus _policy_entitlement_days for a specific reason: the policy function has four early returns, and adding the grant to each one is exactly how a future branch would silently forget it. A test forces three balance reads in a row, because the erasure this design prevents only appears on the SECOND read. entitlement gains a +grants source suffix so a number that came from an administrator is self-describing rather than mysterious. Before/after totals are READ FROM THE BALANCE rather than computed as before+days, because a grant can push an employee past the carry-forward cap - in which case the arithmetic sum states a ceiling they do not have, in the very record an administrator reads to decide whether to grant again. A grant may be NEGATIVE, since the same route is how a mis-keyed one is corrected and the correction stays in the same ledger as the mistake rather than leaving the first entry looking like the last word. Each employee in a batch commits and audits independently and partial success answers 207: one mistyped id in a list of fifty would otherwise cost the other forty-nine their adjustment. reason is required, because HR/Admin added 3 days is not a fact an auditor can use. An archived or blocked employee is refused with the reason - a grant produces a number nobody can spend. The gate is hr_or_admin_required because the requirement names both HR and Admin; @admin_required would have excluded HR, which is the same gate-versus-requirement mismatch this codebase has now found in four places. Two bugs of my own again: six test functions omitted their client parameter, so client resolved to the module-level fixture DEFINITION and every one failed with an AttributeError on .post; and the batch audit assertion queried every LEAVE_GRANT row in the table and expected two, which is only true if it is the only grant test that ran - so it passed in isolation and failed in a full run. It is now scoped to the returned grant ids, which is also the stronger assertion. Matrix 66 IMPLEMENTED / 32 PARTIAL / 5 NOT_STARTED / 1 RETIRED. Unit 305 passed / 2 skipped, browser 23/23, v2.0 gates 113/113 GET + 58/58 write, probe run twice for idempotency.

2026-10-02	FR-LEA-09 (and FR-LEA-02): one working-day function, called from one place. The SRS states the defect and the fix together - the working-day/holiday-deduction function used for leave days, the payroll LOP calculation and the reports 'working days' figure IS the same function, called from one place; v1.0 used three different day-counting rules including a separate Mon-Fri helper. SIX rules were live, not three, and they disagreed. Leave counted (end - start).days + 1, so booking Friday to Monday cost FOUR days of a twelve-day allowance, two of them a weekend the employee never intended to take. Payroll counted attendance_days rows with status Absent OR Half-day, so an employee marked half-present lost a FULL day's pay - FR-JOB-01's classification had already made that distinction and the money threw it away. Reports had no working-day figure at all, reporting days-with-a-login from user_sessions, which is a fifth rule answering a different question. And leave_policy.days_between was a sixth copy of the calendar rule, and the dangerous one: apply reserved working days while reject and cancel gave back CALENDAR days, so every rejected leave silently INCREASED the balance. working_days.py now owns the rule and all six route through it, with a test that walks the AST of app.py and fails if an inline day-count expression or an unexpected call site appears. Working days are PER EMPLOYEE via get_weekly_off_pattern, because this application has no company-wide Mon-Fri week and never did (FR-ATT-17) - a night-shift operator is not off on Saturday, so the v1.0 Mon-Fri helper was wrong for them specifically. Holidays are deducted through _is_attendance_holiday so National applies to everyone and Optional only to an approved opt-in (FR-HOL-03). A range with no working days is refused with a 400 naming the reason rather than recorded as a Pending request reserving zero. The figure is now STORED on the request (Alembic 0010, nullable and deliberately unbackfilled because a request approved under the old rule has no honest value to reconstruct), so approve and cancel move exactly what apply reserved: a holiday added between applying and approving would otherwise make approve release a different number of days, with every audit row still honest, which is the FR-LEA-06 ledger defect reappearing one layer down. FR-LEA-02 also gained the `session` field (Full | First-half | Second-half), which the SRS lists in the create payload: the column existed on the canonical schema with no writer and no reader, so a half-day leave could not be expressed at all and an employee on a four-hour shift had to book a whole day. Six behaviour-changing bugs found along the way, all mine and all recorded rather than quietly fixed: reject_leave had a 5-column SELECT while the new code read index 6, which is an IndexError; leave_policy.cancel read request_row[7] for days when the route's column order puts days at 6 and session at 7, passing the string 'Full' into an INTEGER parameter two frames from the cause; the export route closes its connection in a finally before the row comprehension, so a per-row function call ran on a closed cursor (fixed by reporting the STORED figure, which also guarantees the sheet agrees with the ledger); three test fixtures and two probe flows used dates landing on weekends and were correctly refused; the probe's Monday anchor was defeated by +40 because 40 % 7 is 5, so anchoring to Monday then adding 40 lands on a Saturday; and two hardcoded calendar-day expectations (== 3, == 8) now read the figure from the response, because a shared function called twice can legitimately answer differently the second time and the ledger assertions should check symmetry rather than re-derive a calendar the test does not own. Matrix 65 IMPLEMENTED / 32 PARTIAL / 6 NOT_STARTED / 1 RETIRED. Unit 299 passed / 2 skipped, browser 23/23, v2.0 gates 111/111 GET + 57/57 write, probe run twice for idempotency.

2026-10-02	ToDo list restructured to be SRS-driven, with the requirement coverage section GENERATED from traceability.py rather than maintained by hand. Asked to update the list only, which is when it became clear the document had two problems. First, it answered the wrong question: the task list recorded WHAT WE DID (history) while a to-do list needs to answer WHAT THE SRS ASKS FOR AND WHAT STATE IS EACH IN. Second, and worse, its GATES block was reporting stale numbers - 217 passed against an actual 290, 97/97 GET against 111/111, Alembic head 0005 against 0009 - and nothing failed, because a hand-copied number has nothing to compare itself to. That is the same failure this project keeps finding in traceability rows, committed in the document whose job is to report status. So the new SRS section is derived from traceability.py, whose four tests keep the id set aligned with the SRS and the routes real, which means the coverage figures and the working list cannot disagree with the code. Page 1 is now requirement coverage (104 requirements, 63 IMPLEMENTED, 33 PARTIAL, 7 NOT_STARTED, 1 RETIRED, 61 percent fully implemented) plus a per-module table with a completion bar, and the following pages are the 40 open requirements ordered by the SRS's OWN priority - High first, because a High-priority gap in Documents outranks a Low-priority gap in Analytics regardless of alphabetical order, which is how the SRS ranks them. Each row shows the requirement, its module, its routes, and the NAMED GAP from the matrix rather than a restatement of the title. Extracting that gap well took three attempts and each failed differently, which is worth recording because the tests caught all three. Taking a fixed number of trailing sentences produced credit-then-gap for FR-ATT-06 and mid-sentence truncation for FR-AUD-01. A bare 'not ' signal then matched ordinary English - FR-AUD-01's real gap was being missed because 'whether or NOT it returned anything' tripped it, and word-boundary matching would not have helped because 'not' is a standalone word there too. And a single pass over strong and weak signals let a weak match beat a strong one merely by sitting later in the note: FR-AUD-01's actual gap is 'the row stays PARTIAL only because the SRS also asks for the row to be written via the transactional outbox', an unambiguous phrase, but a later sentence containing 'did not exist' won the walk. So the search is now two-pass - strong phrases first, weak only near the start of a sentence - and all 40 gaps come out between 31 and 199 characters, none opening mid-sentence. Two more findings the tests surfaced: reportlab's paraparser ate '' as an XML tag, so routes rendered as '/api/break-approvals//approve' - a route that does not exist, printed in the one place whose job is to state which routes do. The escaping is extracted into _escape_routes rather than inlined, because the first version of that test re-implemented the same three replace calls and so was only testing itself. And FR-ANL-04's gap is 'the weights are literals in the handler' - a real limitation with no negation, no absence and no 'only', so the extractor fell through to the sentence after it, which reads as a consequence rather than as the gap; 'literal' and 'hard-coded' are now strong signals. One test assertion was wrong rather than the code: it expected RETIRED requirements on the working list, but a superseded requirement (FR-ATT-10) is not open work. Three new tests keep it honest - that the section is generated and agrees with the matrix, that every open requirement extracts a readable gap which names a limitation, and that route patterns survive the markup. Unit 293 passed / 2 skipped.

2026-10-02	FR-NOT-01 and FR-NOT-03: notification email moved onto the outbox, which closes both rows. FR-NOT-01 was PARTIAL because delivery was a direct send_email call on the request thread - an availability defect, not a style preference: SMTP is a network call to a third party and the old code had no timeout at all, so a slow provider held a web worker for as long as it chose. One of the three call sites was the LOGIN path, so a hanging provider would hold the worker meant to be refusing the attempt. All three call sites - the lockout notice, the admin compose endpoint, and the welcome mail on user creation - now enqueue a notification.email event. FR-NOT-03 was PARTIAL because the per-category email column was stored and reported and read by NOTHING: a switch with no circuit behind it. The handler is now the consumer, and it consults the preference before sending. A muted category is retired as DELIVERED rather than failed, and the reason is worth stating: returning False would retry, and if the employee re-enabled the switch during the backoff the mail would then go out, which is the opposite of what they asked for, and it would burn five attempts and dead-letter a notification nobody was ever meant to receive. The preference is read at DISPATCH time rather than enqueue time, so turning a category off after an event was queued does not mail it; the converse is accepted and stated, since such an event was legitimately queued. Two deliberate exceptions, both documented at the call site rather than buried here. A broken preference lookup SENDS anyway and logs, because failing closed would silently drop a possible account-security notice and an unwanted email is recoverable while a silently dropped security notice is not. And the admin compose endpoint forces the send, because suppressing an explicit instruction would leave the admin believing mail went out; it is also a data-exfiltration route by construction, so it stays audited on both paths. The lockout notice forces too, because the SRS pairs the lock with a notification precisely so the login response cannot become a status oracle - a notification the employee could have muted is not the control the requirement describes. Two of my own response fields had to change and the reasoning is recorded rather than the assertion loosened. The admin compose endpoint now answers 202 with an event id instead of 200 and a delivered claim it can no longer make - a 200 saying 'Email sent' from a route that did not send anything is the same defect as send_email returning True for a send that never happened. And user creation reports email_queued and no longer carries email_sent at all: the value was hardcoded True, and reporting False would be no better now, because the route does not know either. What it can honestly say is that the credentials were not delivered by this request, and it names the recovery route so an admin on a no-mail deployment is not left waiting. The request-thread property is asserted structurally, by an AST sweep that fails if any send_email(call site reappears in app.py, because a behavioural test would only catch the regression when a provider happened to be slow. A test helper also had to be built properly: the stand-in outbox row is six columns in the real order, because _payload reads row[4] and an IndexError inside a handler is swallowed by dispatch_once's own except - which would have made the test pass for the wrong reason. Matrix 63 IMPLEMENTED / 33 PARTIAL / 7 NOT_STARTED / 1 RETIRED. Unit 290 passed / 2 skipped, browser 23/23, v2.0 gates 111/111 GET + 57/57 write.

2026-10-02	FR-AUTH-14 / FR-JOB-02: breaks left Active are now auto-closed. The SRS names both duties in one sentence at HIGH priority - 'a scheduled job purges expired reset tokens hourly AND auto-closes breaks Active for more than 12 hours' - and only the first shipped, so a break whose end was never pressed stayed Active INDEFINITELY. The consequences are not cosmetic: that row is what attendance and the payroll loss-of-pay calculation both read, and FR-ATT-09's shift summary adds its open time to the hours an employee appears to have worked, which inflates their hours on paper and is impossible to spot without finding the row. Nothing else in the system ever revisited it. The canonical schema had already anticipated this - breaks.ended_reason exists with the vocabulary documented on it (orphan_timeout|admin_dispose|auto_end_new_break) and NO WRITER anywhere, so a break could be closed four different ways with no way to tell which applied; the column now gets its first value and the compat schema gained it additively. The design decision worth recording is why 12 hours is a THRESHOLD and not a duration: the gap between start_time and the sweep says how long the ROW was open, not how long the break was. An employee who forgot at 11:00 and whose row is swept at 23:00 has not taken a 12-hour break, and recording one would manufacture an absence and a loss-of-pay deduction out of a forgotten button press. So the sweep decides STATUS (this row can no longer be believed to be running) and records DURATION from the break type's own daily limit, capped by elapsed time - the most the break could have been worth. The write is conditional on status = Active, so a repeat pass or a racing pod is a no-op rather than a double notification and a double audit row. Every closure is audited with actor SYSTEM (before/after) and notified to the employee, because the recorded duration is a guess and they are the only party who knows when they came back; FR-ATT-16 admin disposal is how a wrong record is corrected, and an employee who was never told cannot ask. That notification is Attendance, and the taxonomy test forced the decision on the first run as it has three times before - naming the category makes break events PREFERENCEABLE, so an employee who mutes Attendance will not be told their break was closed for them. That cost is real here in a way it was not for BREAK_DISPOSED, because the duration is a guess and the notice is what makes it correctable; they can still see the break in their own record and ask an admin, so muting delays the correction rather than preventing it. Third instance of a shape this codebase keeps hitting: the job is wrapped in an application context because a scheduler thread has none, and audit_log degrades for REQUEST metadata only - without it the audit rows raised, were swallowed by audit_log own except, and silently did not exist. Five unit tests plus a probe flow (attendance(orphan break auto-closed)) that is the only real proof it works on the canonical table: writing ended_reason is the assertion, since a sweep that updated status alone would pass on legacy and produce a row recording nothing about WHY it was closed. Matrix 62 IMPLEMENTED / 34 PARTIAL / 7 NOT_STARTED / 1 RETIRED. Unit 285 passed / 2 skipped, v2.0 gates 111/111 GET + 57/57 write, probe run twice for idempotency.

2026-10-02	Admin-set password: the recovery path that does not need an email server. The gap is real and total - the reset link's only delivery is the outbox and every delivery goes through send_email, so on a deployment with no mail server a forgotten password is UNRECOVERABLE: the employee is told, truthfully and uniformly, that 'if the account exists, a reset link has been sent', and then waits forever. FR-AUTH-08's uniform 202 is the right design and makes this worse, because there is nothing in the response an employee or an admin can react to. Four decisions are not visible in the signature, each a control that could plausibly have been missed. Sessions are closed (DB rows marked closed plus Redis revocation) - a reset that leaves sessions alive is not a reset, because the usual reason for resetting is that the password may have been exposed and the exposed session would survive it. The lockout is cleared, or an admin sets a password, tells the employee to try again, and they fail for another 15 minutes - an admin action that appears to work and does not. Setting your OWN password this way requires your current password, or a hijacked admin session - which may expire - becomes an account the attacker keeps by setting a password only they know; self-service already requires it and this route must not be the way around it. And no password or hash goes into audit_log, which is retained for years. A 409 names the action which would help, and caught while testing, the advice has to use the SAME WORD as the button that does it: the first version told a Blocked user to 'Activate the account' when the control is called Unblock, and distinct from Unlock, which clears a temporary lockout (FR-AUTH-03) - an admin left hunting for an Activate button that does not exist has been told nothing useful. A test of mine asserted the wrong thing and the code was right: I asserted the user_sessions count was zero after a reset, but sessions are MARKED closed, not deleted, and that is the point - the table is the record of hours worked feeding attendance and payroll, so deleting rows to satisfy a security control destroys statutory data. The test now asserts no OPEN sessions and that both rows and their total_hours survive. The email_sent lie one level up: user creation hardcoded email_sent: True, the same defect as send_email returning True for a send that never happened - with no SMTP the employee is created, the response says the welcome email went out, and nobody received it, so an admin has no way to learn the credentials never left. It now reports the real result and names the recovery route. The notification taxonomy test caught the new type on the first run, landing ADMIN_PASSWORD_SET in General; it is Security, the same class as MFA_RESET, because an employee whose password was set by an administrator must be told or their next sign-in demands credentials they never chose and they conclude they were attacked. Fourth time that test has earned its keep. A browser-test defect passed alone and failed in a full run - the sixth instance of test-order interference in that file: page.click on a title selector clicks the FIRST match on the page, and because the search input debounces it reset the password of a DIFFERENT employee ('Second Admin EMP902', created by another test) and failed on the label assertion. The click is now scoped to the row containing the emp_id, and the result is asserted not to mention another employee. Never page.click a selector that matches several rows. The browser test drives the real modal and signs in as the employee with the generated password, because a modal that exists in the template but is unreachable passes every API test, and because the generated password must render OUTSIDE the input it came from, which is cleared after submit - the same mistake that once hid a whole success message inside a form success had just hidden. current_emp_id is now exposed to templates from the context processor, read from the DATABASE actor rather than the session copy, so the panel can tell setting someone else's password from setting my own. A probe write flow auth(admin sets a password) proves it on the v2.0 target, the only thing that would catch a write that only works on the legacy shape: the route writes failed_attempts and locked_until beside the hash, and those are the columns Alembic 0009 added. It reuses the probe's authenticated admin client because EMP001 is in mfa.MANDATORY_ROLES, so a fresh login returns a PARKED half-session and every admin route answers 401 - that cost one debugging round. It targets EMP002, which the lockout flows above deliberately locked, so it also proves the lockout clears on the canonical schema. Unit 280 passed / 2 skipped, browser 23/23, v2.0 gates 111/111 GET + 56/56 write, probe run twice for idempotency.

2026-10-02	SMTP configuration asked about, which turned into three real defects in send_email that would have made a configuration matching a provider's own documentation fail. Port 465 was broken: send_email called starttls() unconditionally, which is correct for 587 and RAISES for 465, the implicit-TLS port most providers document first - so a deployment configured exactly as instructed would have failed every send and the log would have shown a TLS error rather than anything about the port. The transport is now selected from the port (SMTP_SSL on 465, STARTTLS otherwise) with SMTP_USE_SSL to override. An unauthenticated relay was refused: login() ran unconditionally and login('', '') against an open relay raises, so an on-prem MTA that legitimately accepts mail from the host with no credentials produced an authentication error giving no hint that the fix was to send no credentials; login is now attempted only when SMTP_USER is set. A hung relay could stall the queue: smtplib.SMTP was called with no timeout, so a server that accepted the TCP connection then stopped responding held the dispatcher thread open indefinitely - the failure mode is not one lost email but a GROWING QUEUE during a mail outage, turning a mail problem into an application one. scripts/check_smtp.py verifies a real configuration by performing the same handshake the application will: connectivity, TLS negotiation (detecting the 465-vs-587 mismatch explicitly), authentication, then one actual delivery, with --to required so it cannot accidentally mail an employee. Two tests assert the transport selection against a stand-in rather than a live server, because the property is which class the configuration picks and in what order it is driven - a real handshake would test the provider rather than this code. Worth stating explicitly: load_dotenv() is already called at module level in app.py, so .env works under gunicorn as well as flask run. Flask's CLI loads it automatically and people reasonably assume gunicorn does too; it does not, and this project is the exception. Unit 272 passed / 2 skipped.
2026-10-02	FR-AUTH-01: the rate limits made the SRS's own NFRs unsatisfiable. The load test the SRS asks for was never written, so the section 10 numbers were asserted and never measured - and writing the harness immediately found a launch blocker no amount of reading would have. Two per-IP-only limiters, both wrong by an order of magnitude: DEFAULT_RATE_LIMIT was 200 per minute keyed by get_remote_address against a sustained target of 150 req/s = 9,000 req/min, 45x short. The shared-NAT consequence is the blocker: measured with 15 distinct employees behind ONE egress address, 25 reads each, 200 of 375 served and 175 x 429 - the 200 is exactly the per-IP bucket, so the first users' ordinary browsing consumed the whole company's budget. With the SRS's own 500 concurrent sessions behind one corporate NAT that is 0.4 requests per minute per user, and behind a reverse proxy every user looked like one address. After the fix, 375/375 served and 0 locked out - and the same check run against the stashed pre-fix code reproduced the failure, because a check that cannot fail proves nothing. LOGIN_RATE_LIMIT was 20/min against a burst target of 200/min (1,000 logins in a 5-minute shift-start window), refusing 90% of a legitimate shift start before anyone mistyped a password. The fix is the split the SRS already names: the global key is the EMPLOYEE identity once authenticated and the remote address only while anonymous - anonymous traffic is the surface actually worth limiting, being the only one an attacker can hammer without credentials. Keyed on identity rather than session, so ten browser tabs do not buy ten budgets and session multiplication buys nothing. Raising the login limit to 200/min is only safe because FR-AUTH-03 exists: the 20/min address limit was standing in for per-account protection, and lockout.py now locks an employee after 10 consecutive failures in 15 minutes. A test of mine asserted the wrong thing and the correction is recorded rather than the assertion loosened: the first ratchet compared DEFAULT_RATE_LIMIT against 9,000 req/min and failed, but the COMPARISON was the error - 150 req/s across the API is an aggregate and the limit is now per employee. The old 200/min was broken precisely because it was an aggregate number applied as a per-user one. The test now checks the per-employee floor (the admin dashboard polls every 5 s, 12/min by itself, plus panels) and that 500 sessions clear the aggregate target. A harness flaw would have produced a fake finding too: loadcheck first reported a 17% error rate because its endpoint list included the admin-only dashboard-stats and it counted the correct 403 as a server fault. Measured after the fix at 80 req/s: p95 181ms, p99 226ms, 0.000% errors, 0 lockouts, against SRS targets of p95 under 300ms, p99 under 800ms and errors under 0.1%; before the fix the same run produced 141 lockouts and 13% errors. ops/load/load.js is the k6 scenario the SRS names, and it refuses to run against an MFA-enrolled account rather than measuring a wall of 401s, because FR-AUTH-11 makes a second factor compulsory for Admin, HR and Finance. Matrix moves FR-AUTH-01 to IMPLEMENTED: 60 IMPLEMENTED / 36 PARTIAL / 7 NOT_STARTED / 1 RETIRED. Unit 269 passed / 2 skipped, browser 22/22, v2.0 gates 111/111 GET + 55/55 write, probe run twice for idempotency. A sixth cleanup-helper failure, and the fix that ends the class: removing the temporary load-test employees raised an FK violation from leave_balance, and this file has listed the referencing tables wrongly three times, so the cleanup now discovers all 35 of them from information_schema instead of listing them.

2026-10-02	Backup + restore drill (SRS 10: daily full backup, WAL archiving, 30-day retention, quarterly restore drill - none of it existed). scripts/backup.py provides backup, list, restore and verify; verify is the one that matters because it restores into a scratch database and then runs the application's OWN gates against it (check_cc_rules, cutover_preflight), since restoring is the easy half and a backup that restores into a database the app would refuse to run is not a backup. It also checks a row count, because every gate passes on an EMPTY database - that is the specific way this class of check lies - so users=0 fails and so does a dump with no users table at all, which is what a backup of the wrong database looks like. Both directions were run: a real backup verified green (exit 0) and a deliberately empty dump verified red (exit 1) with a diagnosis rather than a crash, because a drill that cannot fail is not a drill. WAL archiving is deliberately NOT implemented and says so: PITR needs a WAL destination and a retention policy belonging to whoever owns the storage, and a script that pretended to do it would be worse than one that declines. The container fallback is real rather than a comment - the dump is STREAMED through docker exec because a host path is not visible inside the container, and the DSN endpoint is rewritten to the container's internal port. Three connection bugs came out of writing this; the instructive one is that rebuilding netloc from the host alone THREW THE CREDENTIALS AWAY, surfacing as 'role root does not exist', a failure that looks like a database problem and is not one. Two DSNs are used on purpose: psql inside the container needs the internal endpoint, the Python gates run on the host and need the published port. Cron runs at 03:17 rather than 03:00 so a full dump does not compete with the 02:05 attendance finalisation, and the quarterly drill exits non-zero so an unwatched run still leaves a record of having failed. backups/ is gitignored: a dump is a full copy of every employee record.
2026-10-02	Go-live review: three of the five blockers turned out to be mine to fix rather than operator tasks. (1) send_email returned TRUE when SMTP_HOST was unset, logging 'would send'. Every delivery path goes through the outbox, so with the default configuration the event was marked delivered, the outbox monitor showed nothing wrong, the queue drained cleanly - and no human ever received a password reset link, while the employee had already been told the link had been sent. Found by auditing work done WITH the behaviour: FR-AUTH-08 was rebuilt this session so the response carries nothing, which is only an improvement if the link is actually delivered, and my own test asserted 'the queue drains', which was the bug. Reporting success for a send that did not happen is worse than having no email feature at all. Now a failure, so the event retries and dead-letters. The 202 stays uniform and cannot mention SMTP without becoming an enumeration oracle. (2) SRS 11.3 names Flask-Talisman, CSP with no inline scripts, nosniff, X-Frame-Options DENY, Referrer-Policy and HSTS, and the application shipped NONE of them - grep found only SESSION_COOKIE_SECURE. CSP is a module constant so the test asserts the configuration the app boots with rather than a hand-copied duplicate that would only check itself; script-src carries no unsafe-inline per the SRS's own parenthetical and the single relaxation is style-src, narrow and commented. frame-ancestors 'none' and X-Frame-Options DENY are both sent because CSP is not honoured everywhere the SRS targets. Disabled outside production deliberately: a CSP that fires on every local page load is one people learn to ignore. The test boots a real production app in a subprocess, because re-initialising Talisman onto the suite's app registered two after_request handlers that interfered and the tempting fix was to loosen the assertion until it passed. (3) FR-JOB-05: 'start in the gunicorn master' is right for one instance and silently wrong for several, since every pod has its own master, so an N-pod deployment ran every cron job N times - unnoticed because duplication was mostly absorbed by idempotency built for other reasons. scheduler_leader.py is a Redis lease with token-fenced renewal via Lua compare-then-extend (a GET/PUT pair would let a stale leader overwrite the new leader's token), and a lost lease shuts the scheduler down. A configured but unreachable Redis refuses to start the scheduler: _redis() returns None for both 'not configured' and 'unreachable', so without an explicit check a broken lease store falls through to the legacy heuristic and every pod starts its own scheduler - the exact bug, reached by the most innocent-looking route. The SRS chaos test is literal: three competing OS processes race and exactly one wins, with three distinct identities. Also: both password-reset routes had a hardcoded 5/min limit neither suite lifted - the second LOGIN_RATE_LIMIT gap - now env-overridable; and a new /api/health reporting degraded rather than unhealthy when SMTP is unconfigured, when outbox events have dead-lettered, and which instance holds the scheduler lease, because a 200 only proves the process is up. Matrix now 59 IMPLEMENTED / 37 PARTIAL / 7 NOT_STARTED / 1 RETIRED. Unit 265 passed / 2 skipped (the leader chaos test skips without REDIS_URL and passes with it), browser 22/22, Redis 10/10, v2.0 gates 111/111 GET + 55/55 write.

2026-10-02	Group 3 of the audit: every mutating handler now writes an audit row except one deliberate exemption, so FR-AUD-01's 'every mutating action' holds in substance and the row stays PARTIAL only because the SRS also asks for the row to be written via the transactional outbox and audit_log() writes straight to the table. Nine handlers fixed: payroll run creation, asset issue, salary structure, import job cancel/run, the manual offboarding access-revocation pass, the break-approval request, and the regularization request - plus return_asset, which had the always-200 lie for the THIRD time: it updated unconditionally and answered 200 {'message': 'Asset returned'} whether or not it returned anything, including for an id that does not exist. That one has a custody consequence - an admin told a laptop came back when it did not is how one goes missing quietly - so it is now 404 for unknown and 409 naming the date it was already returned. The exemption is mark_notifications_read: a read receipt on the caller's own notifications, where a row per click is noise that makes the real entries harder to find. It stays on the ratchet list rather than being deleted, because a name there is a claim someone has to re-justify whereas an absence is indistinguishable from having been forgotten. Two design notes worth keeping: salary AMOUNTS are deliberately not copied into the audit row, because the log is retained for years and must not become a second weaker copy of the payroll tables - the audit records that a structure changed and for which period, and salary_structures remains the record of what it is. And the ratchet now holds one name, having started at twenty. Two of my own bugs in this slice, both caught immediately: the assets insert I wrote had nine parameters for ten placeholders because return_date is still a placeholder and only status is a literal - which is exactly the class of defect the edit existed to remove - and a break test leaked a notification row into an unrelated test, the fourth fixture leak in that file. Unit 258 passed / 1 skipped, browser 22/22, Redis 10/10, v2.0 gates 109/109 GET + 55/55 write.
2026-10-02	Group 2 of the audit: two more bare INSERT INTO VALUES (...) in documents_api and dependents_api, and three exemption calls. Neither bare insert is a live failure - both tables have the same column count and order on legacy and on v2.0 public today - which is exactly why it was worth fixing: the bug is invisible until someone adds a column, which is what happened with POST /api/goals (ten placeholders, nine columns, 500 on every backend) and add_holiday (five placeholders, v2.0's six, every value mis-targeted). Both times the boot seed kept working because the seed names its columns, and both of these had a seed insert two functions above writing the column list out. The guard is a test rather than the fix: it parses db/postgres_schema.sql for each table's declared columns and asserts the route's list still matches, so adding a column without updating the route fails there instead of at the public flip. Three 'should this audit?' calls did not go as predicted: mark_notifications_read is deliberately EXEMPT (a read receipt on the caller's own notifications is a row per click, and noise that makes real entries harder to find is the opposite of the point) and the exemption is recorded in the ratchet with its reason; admin_outbox_dispatch is NOT exempt because a dispatch sends email to employees and 'who forced the queue out and what did it deliver' is the question an operator pressing it at the wrong moment needs answered; send_notification_email is NOT exempt and is audited on BOTH paths, because an admin route that mails any address with any body is a data-exfiltration route by construction and recording only the successes would hide the attempts that matter. add_holiday audited nothing while its edit already did, so a holiday could be added to the company calendar with no record and then edited with one. Ratchet 15 -> 10 handlers (14 gaps + 1 documented exemption). One browser run failed 2 tests and the cause was self-inflicted: the probe and the Redis suite were run concurrently with the browser suite against the same PostgreSQL instance, and two timing-sensitive tests failed; re-run alone it is 22/22. Investigating it surfaced two real weaknesses, both fixed - _login's MFA panel window was 5s, which on a loaded machine reads as 'no MFA' and then times out waiting for a dashboard that never arrives, and test_admin_edits_user_permissions used three fixed sleeps before asserting the permission grid had rendered. Fourth and fifth instances of sleep-vs-signal in that file. Unit 256 passed / 1 skipped, browser 22/22, Redis 10/10, v2.0 gates 109/109 GET + 55/55 write.

2026-10-02	Break-lifecycle audit (FR-ATT-03/06/16): four requirements whose notes asserted behaviour the handlers did not have, in six handlers. FR-ATT-06's gate was @admin_required while the SRS says the employee's MANAGER (or HR/Admin) may approve - so a Team Leader could not approve their own report's break. That is the THIRD instance of that bug here, after FR-EXP-03 (Approved -> Paid unreachable for Finance) and FR-PERF-01 (goal rating unreachable for the reporting manager), and all three were recorded as implemented while the decorator made them impossible. Fixed by reusing the reporting_line_required gate from the goals work rather than writing a fourth. The same row claimed a conditional write that was unconditional (approve filtered on Pending in the SELECT but the UPDATE had no status guard, so two approvers both won), an audit that did not happen, and a notification that was never sent. reject_break also had the always-200 lie already fixed in regularization - it answered {'message': 'Break rejected'} whether or not it rejected anything; two routes in two unrelated slices carrying the same bug is the argument for sweeping siblings rather than fixing one at a time. FR-ATT-16 claimed 'an audited reason' for a route with neither an audit row nor a reason parameter; the reason is not cosmetic, since disposing ends ANOTHER employee's break and so shortens their attendance and their pay, and a blank reason is refused rather than defaulted, lands in the audit row and is sent to the employee. FR-ATT-03 claimed 'audited' and end_break audited nothing; start_break did not either despite silently auto-ending a previous break. One deliberate earlier decision had to be reversed: notifications.py mapped no rule for BREAK_* events on the reasoning that a rule to FALLBACK is a no-op that reads like data - true only while nothing emitted one. FR-ATT-16's notification now does, the emitted-types test forced the decision exactly as its author predicted, and break events are named Attendance (a fourth documented extra). Not a free win: naming it makes break events preferenceable, so an employee can now mute them where the catch-all delivered them unconditionally; the module's own principle settles it. PUNCH still emits nothing and still falls to the catch-all, and the test asserting the old behaviour was rewritten with the reasoning rather than quietly flipped. The audit ratchet caught five of its own names going stale the moment they started auditing - 20 handlers down to 15. Unit 253 passed / 1 skipped.
2026-10-02	Audit pass over the IMPLEMENTED rows, after three requirements in a row turned out to be recorded as done and not be. Four findings. FR-AUTH-07's row claimed 'unauthenticated gets 302 for a page and 401 for JSON', but _wants_json checked is_json and the /api/ path and NOT the Accept header the SRS names, so GET /dashboard with Accept: application/json answered 302 to the login page - HTML for a caller that asked for JSON, which then follows the redirect and cannot parse what it got. Identical to the multipart failure fixed in the FR-HOL-01 slice, one trigger earlier; now honoured, with a combined Accept still redirecting so browsers are unaffected. FR-REG-03's approval route had its success return OUTSIDE the if that did the work, so every outcome was a 200: a non-existent request, an already-decided one, and - the one that mattered - rejecting an already-approved request, which answered {'message': 'Rejected'} while the row still said Approved, so a client checking status_code believed a decision it had not made. It also audited nothing, so an attendance correction feeding payroll left no trail. Now 404 for unknown, 409 naming the state found, conditional write, before/after audit. FR-AUD-01 was downgraded IMPLEMENTED -> PARTIAL: 'every mutating action' was false. Document deletion wrote no audit row while document DOWNLOAD did, which is backwards, since deleting removes the row and the file; deleting a dependent erased policy-classified PII with no trail. Both fixed. The other 20 unaudited mutating handlers are named in the row and held by a ratchet test that fails if a new mutating route skips the audit log and fails if a stale entry is left behind. FR-AUTH-12 and FR-USR-06 notes corrected: the CSRF token is a per-session synchroniser, not the double-submit cookie the SRS describes, and enforcement is app-wide rather than /api/-scoped - both stricter, but the note described something the code does not do. FR-USR-06 names bank details among the identifiers to scrub and no such column exists, so that clause is vacuous; a test now fails if a personal column is added without being taught to the eraser. Matrix is now 58 IMPLEMENTED / 37 PARTIAL / 8 NOT_STARTED / 1 RETIRED.

2026-10-02	FR-AUTH-08/09 password reset: FR-AUTH-08 was recorded IMPLEMENTED and was the worst control failure in the application. /api/forgot-password answered 404 {'error': 'No matching user found'} for an unknown account and 200 *carrying the working token* for a real one - the control inverted rather than weakened, since a caller could confirm any employee ID and, if the email matched, obtain a credential without ever touching the account. Four request shapes now answer a byte-identical 202 with the SRS's own sentence and no token, and the token moved to the outbox, which is where the SRS puts it and what makes an empty response possible; the token row and its delivery event commit together (CC-09). FR-AUTH-09's row was wrong twice: it said the token was stored unhashed, which was half true and arguably worse, because hashing existed on the write path while /api/reset-password looked tokens up as token IN (raw, digest) to accommodate two plaintext tokens the boot seed wrote - so a database read still yielded two usable credentials on every fresh database; and it said the expiry should be 24 h, which is wrong - that token belongs to FR-USR-02's welcome email, while FR-AUTH-09 specifies 1 h, which the code already used. Also added the half that mattered: ALL other live tokens for the account are invalidated on a successful reset, because otherwise an attacker who requested their own reset while a legitimate one was live kept a working credential after the legitimate user reset theirs, and the single-use write is now conditional on used = 0 so two concurrent replays cannot both win. The emailed URL did not exist either - the SRS points at /reset-password?token=... and that was a 404, so the journey was reachable only by calling the API and reading the token out of the response; the page now exists and never interpolates the token into the HTML. A browser test caught a real UI bug in it: the success message was inside the form, and success hides the form, so the confirmation was hidden with it. Matrix is now 59 IMPLEMENTED / 36 PARTIAL / 8 NOT_STARTED / 1 RETIRED, and the generator's closing section now warns that the rows worth auditing next are the ones asserting something is finished - FR-AUTH-02 and FR-AUTH-08 were both recorded IMPLEMENTED and both were not, and neither was found by reading the code for its own requirement but for a neighbouring one.
2026-10-01	FR-AUTH-03 account lockout, and the FR-AUTH-02 hole it exposed: the SRS is four numbers - 10 consecutive failures within 15 minutes lock the account for 15 minutes and notify the user by email - and lockout.py implements exactly those, asserted by a test so a threshold nobody agreed to cannot hide in a constant. A successful sign-in breaks the streak and the window slides, because without both, four typos spread over a week lock an employee out and a lifetime counter trains people to write passwords on sticky notes. An admin can clear a lockout without touching users.status: a lockout is a temporary consequence of wrong passwords while Blocked is a sanctioned account state, and folding them together writes an HR record against a fifteen-minute nuisance. Deliberate deviation, recorded in the matrix: the SRS flow diagram puts the counter in Redis and it is stored on users instead, because this app treats Redis as optional and a lockout that silently stops existing when Redis is unreachable has failed open rather than degraded - the same rule that put the password-policy corpus offline. Alembic 0009 adds three additive columns, no backfill, and no index, since nothing queries by locked_until. The consequential find was reading the login handler to wire it up: FR-AUTH-02 was recorded as IMPLEMENTED while the route answered two different 401 messages and two 403s ('Account is blocked', 'Login is not allowed') - the exact state leak the requirement forbids, and adding a lockout would have added a third. All six refusal paths are now one 401 {'error':'invalid_credentials'}, the password is verified before any state is considered so timing says nothing either, and a test asserts all six answers are byte-identical. The obvious feature - a 'locked, try again in 14 minutes' banner on the login page - was built and then removed: it is the enumeration channel the requirement forbids, which is why the SRS pairs the lock with a notification instead. Matrix is now 58 IMPLEMENTED / 37 PARTIAL / 8 NOT_STARTED / 1 RETIRED.

2026-10-01	FR-AUTH-11 multi-factor authentication: the largest single gap the traceability pass found - mfa_credentials was in the canonical schema with an encrypted-secret column and no code anywhere that read or wrote it, no enrolment, no challenge, no gate. mfa.py now owns TOTP (RFC 6238), Fernet encryption under a dedicated MFA_ENCRYPTION_KEY (a missing key is a 503 refusal, never plaintext storage, mirroring ANONYMISATION_SALT), the pending-login state machine and the attempt cap. Enrolment is two-phase: /api/mfa/enrol writes the row with enabled=0 and only a valid code at /api/mfa/confirm promotes it, so a stolen password cannot enrol an attacker's own authenticator against the account. The password step parks the identity in session['mfa_pending'] and deliberately does NOT set session['emp_id'], so a half-authenticated session is refused by every existing gate by construction rather than by each route remembering to check. Compulsory for Admin/Super Admin/HR/Finance, self-service opt-in for everyone else; a mandatory role cannot disable its own factor and re-enrolment while enabled is a 409. Five wrong codes abandon the parked login; one step of clock drift is tolerated; the pending state expires after ten minutes. Recovery is an audited Admin reset only - no recovery codes, which is the weaker answer the SRS allows, so the reset answers identically whether or not the target was enrolled (it cannot be used to find out who is protected) and writes a before/after audit row. Two real bugs found while validating: the login page built the finishing endpoint as '/api/mfa/' + step, pointing the enrolment submit back at /api/mfa/enrol - which mints a new secret - so the code the user had just read stopped matching and the page navigated on without verifying anything; and signing in as a second account in the same browser kept the first account's session, because the cookie survives and _mfa_subject prefers session['emp_id'], so the enrolment and challenge both acted on the wrong person. Fixing that also closed a pre-existing leak where a test blocked the seeded EMP002 and never restored it. Matrix is now 57 IMPLEMENTED / 37 PARTIAL / 9 NOT_STARTED / 1 RETIRED.
2026-09-30	FR-HOL-01/02 holiday calendar: the duplicate rule was a SELECT in the route, which is a message and not a rule - two concurrent adds both passed the check and both inserted. It is now a unique index, and the obvious constraint would have been wrong: a plain UNIQUE(name, holiday_date, location) accepts duplicate org-wide holidays because NULL is distinct from NULL in SQL, so the index is on COALESCE(location, '') and the application builds the identical triple. Added the missing update, year-to-year copy that skips rather than shifts a 29 February holiday, CSV import/export, an iCal feed that uses DTSTART;VALUE=DATE and folds every line to the 75-octet RFC 5545 limit, and a delete that is refused while opt-ins reference the holiday. The consequential find: the role gates decided 'is this an API call?' with request.is_json, which is False for a multipart upload, so every admin-gated upload route answered a non-admin with a 302 to dashboard HTML that a fetch client follows and cannot parse, with a 200 status. app._wants_json() now keys on the path as well. Matrix is now 56 IMPLEMENTED / 36 PARTIAL / 11 NOT_STARTED / 1 RETIRED.
2026-09-30	FR-HOL-03 optional-holiday opt-ins: a defect rather than a feature. holiday_optins existed in the canonical schema, init_db created it on the compatibility shape, and nothing ever wrote to it. An Optional holiday is an attendance holiday only for an employee with an Approved opt-in, no employee could ever obtain one, and the nightly FR-JOB-01 finalisation recorded the seeded Diwali as Weekly-off and would have said Absent on Christmas - an implemented High-priority requirement producing a wrong answer. Employee requests, HR approves or rejects from a queue, the owner may withdraw. The canonical schema also contradicted the requirement and itself: uq_optin was a plain UNIQUE (emp_id, holiday_id), a constraint on one request ever rather than one active request, which made opt-out irreversible, and it contradicted the partial uq_active_offer_candidate four lines above it. Alembic 0006 replaces it with the partial index. Also found a latent bare-VALUES insert in add_holiday (five placeholders against v2.0's six-column table) - the same shape that made POST /api/goals return 500 forever - and a latent VARCHAR(32) limit on alembic_version.version_num that fails at the version stamp rather than at the migration. Matrix is now 54 IMPLEMENTED / 38 PARTIAL / 11 NOT_STARTED / 1 RETIRED.
2026-09-30	FR-LEA-05 leave cancellation: the SRS asks for one sentence and there was no cancel route at all, so a Pending leave request reserved days against the employee's balance and nothing could ever give them back - a request that changed its mind silently reduced their remaining leave for the year. The decision and the reversal are leave_policy.cancel's, together, because the ledger effect differs by state: Pending releases the reservation, Approved takes the days back out of used_days, and releasing in that case would leave the balance understated with no way to detect it. The day count is the same expression the apply and approve paths use, so the reversal matches the reservation it undoes. Also fixed a latent test-isolation bug: the leave cleanup deleted the user before the rows referencing it, so the user survived a foreign-key error and the next test to reuse the id failed somewhere unrelated. Matrix is now 53 IMPLEMENTED / 39 PARTIAL / 11 NOT_STARTED / 1 RETIRED.

2026-09-30	FR-TKT-03/04 tickets: FR-TKT-03 asks for defence in depth on the visibility rule, and the list and detail view had it - but the two write paths did not. An unrelated employee's list was empty, the detail view returned 403, and POST /api/tickets//comment returned 201. update_ticket_status had neither a visibility check nor a state machine, so any authenticated user could move any ticket to any state and close anyone else's. tickets.py now holds one can_view rule for all four paths and the strict Open -> In Progress -> Resolved -> Closed chain, with Reopened as a real status: a Closed ticket reopens when its reporter comments within seven days, and an older closure stays closed. Assignment is audited, which FR-TKT-04 asks for and nothing did. The assignee can now see the ticket they were given, which the old rule prevented. FR-DOC-03 folded in: document downloads are audited. This slice also caught a bug in my own patch - jsonify(body, exc.status) returns 200 with a 409-shaped body, so every refused transition read as a success. Matrix is now 52 IMPLEMENTED / 40 PARTIAL / 11 NOT_STARTED / 1 RETIRED.
2026-09-30	FR-PERF-02 review integrity: the SRS has two sentences for this requirement and neither was enforced - submit requires the assigned reviewer (Appendix A-18, recorded as a v1.0 gap) and 360 feedback reviewer cannot be the subject. submit_review had no reviewer check at all, a self-review could be opened because both ids came from the body unexamined, a signed review could be reopened and rewritten because the write was unconditional, and anyone could rate themselves five stars. reviews.py owns the rules, and check_submit deliberately gives HR and Admin no bypass: the rule exists to stop a review being signed by somebody who did not write it. A self-review is refused at creation, the rating is bounded, Submitted is terminal with a conditional write, and the before/after is audited. The probe is also idempotent across runs now - its reset-password flow used to change EMP002's password permanently. Matrix is now 51 IMPLEMENTED / 41 PARTIAL / 11 NOT_STARTED / 1 RETIRED.
2026-09-29	FR-PERF-01 goals: POST /api/goals had never worked - a bare INSERT INTO goals VALUES with ten placeholders against a nine-column table, so every goal creation returned 500 on every backend. The seed used an explicit column list, which is why the seed worked and the create path did not, and no test or probe flow created a goal. PUT /api/goals/ was @login_required with no ownership check, so any authenticated user could rewrite any goal by guessing a sequential id, and could set status to skip rating. The rating enforced neither half of the SRS rule, and @admin_required excluded the reporting manager the requirement names - the same gate bug as the expense slice. goals.py now owns EDITABLE_FIELDS (no status, no rating), the edit ownership rule, and a rating that requires the reporting manager or HR/Admin and refuses the owner, with a conditional write on Active. A new reporting-line gate admits managers. Matrix is now 50 IMPLEMENTED / 42 PARTIAL / 11 NOT_STARTED / 1 RETIRED.
2026-09-29	FR-AUTH-10 password policy: a 10-character minimum (Appendix A-01 calls 6 a defect) plus a breach-corpus check, enforced wherever a password is set, with no complexity rules and no expiry by design per NIST SP 800-63B - a test parses the module's AST to keep them out. The corpus is offline by default (a check that fails when the network is down does not exist) with the HIBP k-anonymity range query behind HIBP_URL. Leet variants and known-password-plus-suffix are caught, and the rejection message is generic so it is not an oracle for confirming a guess. The shared default of pass123 is gone: a user created without a password now gets a generated compliant one returned once. Matrix is now 49 IMPLEMENTED / 43 PARTIAL / 11 NOT_STARTED / 1 RETIRED.
2026-09-29	FR-EXP-03 expense claim state machine: following the traceability matrix into expenses_api found three defects - an admin could approve a claim they had filed themselves, a claim could jump Pending to Paid with no approval, and a paid claim could be moved back to Pending - plus the CC-10 impersonation hole where emp_id came from the request body. expenses.py owns a strict transition table, blocks self-approval, gives Paid to Finance/Admin only (Appendix A-11), requires a rejection reason, and writes every transition as a conditional UPDATE with a before/after audit. The gate was the other half: admin_required excluded both the claim owner's manager and Finance, making Approved to Paid unreachable by the role the SRS names, so a new expense_actor_required gate admits manager, HR, Finance and Admin, and Finance now holds the expenses module. The matrix also corrected itself: the previous claim that FR-DOC-02 validated the file extension rather than the content was wrong - the upload route does sniff the magic number and reject EICAR. Matrix is now 48 IMPLEMENTED / 43 PARTIAL / 12 NOT_STARTED / 1 RETIRED.
2026-09-29	SRS traceability matrix: traceability.py maps all 104 SRS requirements to the routes that implement them, and docs/TRACEABILITY.md is generated from it. Four tests keep it honest - the id set must match the SRS, every route named must exist in the live url_map, an IMPLEMENTED row with no route must explain itself, and a PARTIAL row must name its gap. Verdicts: 47 IMPLEMENTED, 44 PARTIAL, 12 NOT_STARTED, 1 RETIRED. The largest gap found is FR-AUTH-11 MFA: mfa_credentials exists in the canonical schema with an encrypted secret and no code reads or writes it. Same shape for approval_delegations, notification_preferences and holiday_optins. Two security PARTIALs verified by reading the handlers: FR-DOC-02 validates the file extension rather than the content (no malware scan, a renamed .exe passes), and FR-EXP-03 has no self-approval block so an employee can approve their own claim.

2026-09-29	Server-side session coverage: CI re-ran the whole unit suite with REDIS_URL set, but no test in the suite referred to the session store, so the step passed identically whether the backend was Redis or the app had silently fallen back to signed cookies - which is what the first run of the new tests found. tests/test_redis_sessions.py is now its own pytest process (the backend is chosen when app is imported) and asserts the opaque cookie, the Redis payload and TTL, session persistence, logout deleting the key, block and archive revoking the session while an unrelated one survives, re-blocking after an unblock, CSRF still enforced, and both fallback paths. The duplicate 122-test CI step is replaced by these 10 tests.
2026-09-29	FR-LEA-08 monthly leave accrual: monthly_leave_grants was in the canonical target and unwritten, so an accrual rate was collapsed into a flat annual ceiling at assignment time. The rate-driven entitlement is now what the employee has earned so far (floor(rate x months elapsed) across the months the assignment was in force), posted one row per elapsed month by a monthly cron job, POST /api/accrual/run, or the Accrue now button in the leave-policy modal - all three idempotent. The entitlement is derived and the grant is the record, so a missed run cannot leave anybody with no leave; granted_by is NULL because it is a foreign key to users(emp_id) in v2.0. entitlement_days now takes the balance year, and a year the policy never reached keeps the published default. DuckDB 126 passed / 6 skipped; PostgreSQL 131 passed / 1 skipped; Playwright 21 passed; probe 97/97 GET + 45/45 write.
2026-09-28	FR-USR-15 PII completion: policy.PII_FIELDS is now a per-entity map, so a candidate's email and phone are withheld from /api/candidates and /api/offers unless the actor holds pii_reveal, each record carries pii_revealed, and a permitted read writes one PII_REVEAL audit row. A candidate's name is never withheld (a recruiter must know whose record they are editing) and dependents are classified but have no cross-employee read path. Also made the scheduler registration testable without starting a scheduler thread, which is what made the DuckDB unit suite intermittently fail. DuckDB 121 passed / 6 skipped; PostgreSQL 126 passed / 1 skipped; Playwright 21 passed; probe 97/97 GET + 44/44 write.
2026-09-28	FR-USR two-person anonymisation: propose then confirm-by-a-different-approver, applied by the system, archived accounts only, with a dry-run plan, a required ANONYMISATION_SALT, dependents erased and the statutory rows kept, and the subject's own audit history scrubbed by value substitution (including historical values, so a rename cannot leave the old name behind). Trade-offs taken on the operator's behalf - keeping emp_id as the seven-year join key and leaving free text - are recorded in docs/ANONYMISATION.md. Also added an on-demand import run route and made the DuckDB browser fixture scheduler-free. DuckDB 116 passed / 6 skipped; PostgreSQL 121 passed / 1 skipped; Playwright 21 passed; probe 97/97 GET + 44/44 write.
2026-09-28	FR-USR-04 background import jobs: POST /api/users/import now returns 202 with a queued job (5 MB / 5000-row caps, streaming validation, idempotent retry), a dispatcher claims one job per tick with a conditional status transition so several workers cannot collide, progress is published every 25 rows, the outcome is audited and the stored upload is deleted when the job settles. Added the import_jobs table via revision 0004_import_jobs plus the compat DDL, job status/history/cancel routes and a polling UI. Also fixed audit_log, which silently dropped every scheduler-thread write (the nightly ACCESS_REVOKED rows), and made the global rate limit overridable so a full suite no longer trips it mid-run. DuckDB 111 passed / 6 skipped; PostgreSQL 116 passed / 1 skipped; Playwright 20 passed; probe 96/96 GET + 44/44 write.
2026-09-28	FR-LEA-06/08 policy-derived leave balances: the entitlement now comes from the effective leave_policy_assignments (accrual rate x 12, capped by carry-forward) and falls back to the published defaults, so deriving changes nothing until a policy is assigned. The apply path always enforces (it used to skip employees with no row) and reserves the days while a request is pending, which is the first write to the reserved column and closes the double-spending hole. Added an audited policy API plus an admin modal. Also fixed the intermittent DuckDB browser-suite flake (DuckDB attaches a file once per process, so the dev server runs single-threaded there). DuckDB 107 passed / 6 skipped; PostgreSQL 112 passed / 1 skipped; Playwright 19 passed; probe 94/94 GET + 44/44 write.
2026-09-28	FR-USR-15 scope + PII: no handler branches on the session role copy any more (test-enforced), the company-wide list split became policy.can_view_all (CC-11 scope), the dashboard variant follows sees_admin_surface(), and pii_reveal is now enforced and audited: GET /api/users//pii is the only cross-employee personal-data read and every such reveal writes a PII_REVEAL audit row. DuckDB 102 passed / 6 skipped; PostgreSQL 107 passed / 1 skipped; Playwright 18 passed; probe 94/94 GET + 44/44 write.
2026-09-28	FR-USR-15 enforcement wiring: the four role gates now narrow with policy.can() (the role/department check stays the outer gate, so an empty override table is a no-op), every gated view is mapped to a module, and the navbar is derived from the same gate/module tags so a link exists only when the route would admit the user. The HR-department grant is modelled explicitly. Drift removed (Finance no longer sees 403ing links, Super Admin sees the admin links) and /admin/leaves is now reachable. DuckDB 97 passed / 6 skipped; PostgreSQL 102 passed / 1 skipped; Playwright 17 passed; probe 94/94 GET + 44/44 write.

2026-09-28	FR-USR-09 permission policy implemented: policy.py owns the 27-module role matrix with deny-beats-allow per-user overrides, GET/PUT /api/users//permissions does a full replace with an audit diff and an anti-lockout guard, and the admin user list gained a permissions modal. Empty overrides reproduce the role defaults exactly, so no existing authorization outcome changes; decorator/navbar enforcement is the next slice. DuckDB 90 passed / 6 skipped; PostgreSQL 95 passed / 1 skipped; Playwright 17 passed; public probe 94/94 GET + 44/44 write.
2026-09-28	FR-USR directory contract implemented: GET /api/users caps per_page at 200 and sorts through an allow-list (400 on invalid input), POST/PUT validate the employee ID, email, role, department and status with case-insensitive uniqueness, PUT became a true partial update with a before/after audit diff, and the CSV import now validates every row and reports skipped rows. Admin UI mirrors the contract; the public probe stays green.
2026-09-25	Phase 5 cutover rehearsal completed: frozen-source ETL accepted a missing post-v1.0 payroll_approvals table, reconciled Phase-1/Phase-2 data, applied CC-05 cleanup, stamped head, passed read-only preflight and 94/42 public probe, and kept identity sequences ahead after boot seeds and authenticated GET/write smoke. Traffic switch remains a maintenance-window operation.
2026-09-25	FR-ATS/FR-ONB/FR-OFF corrected lifecycle implemented: guarded ATS state machine, 100% offer splits, atomic accepted-offer conversion, signed pre-boarding tokens and real-file validation, five-step guarded onboarding, parallel offboarding with F&F; maker-checker, and IST LWD access revocation, plus lifecycle hardening (strict split precision, ETL preservation, session revocation, encrypted credential delivery, and document authorization). DuckDB 77 passed / 5 skipped; PostgreSQL 81 passed / 1 skipped; Playwright 16 passed; public probe 94/94 GET + 42/42 write.
2026-09-24	FR-PAY-06 maker-checker payroll implemented: Draft → Submitted → Approved → Finalized, Finance/Admin authorization, self-approval rejection, payroll_approvals trail, adjustment-run reference, Finance UI access, and public probe coverage. DuckDB 64 passed / 5 skipped; PostgreSQL 68 passed / 1 skipped; probe 86/86 GET + 30/30 write.
2026-09-24	FR-JOB-01 attendance finalisation implemented: nightly per-shift-date classification, employee weekly-off patterns, holiday/leave precedence, transactional attendance_days replacement, calendar output, and public identity-key probe coverage. DuckDB 64 passed / 5 skipped; PostgreSQL 68 passed / 1 skipped; probe 86/86 GET + 30/30 write.
2026-09-24	Service-layer rewrite inc 2/3 completed: shift_assignments, explicit v2.0 INSERTs, BOOLEAN UPDATE parameter coercion, and binary payroll bank-file export; the clean public-flip probe is fully green.
2026-09-23	CC-07 idempotent writes done: @idempotent decorator on 9 POST routes, idempotency_keys DDL + hourly purge, 6 unit tests green (DuckDB 40 / PG 43 / PG+Redis 43), probe replays on pure v2.0 JSONB -> 86/86 GET + 12/12 write. Next: service-layer rewrite.
2026-09-23	TODO list created. Next task slated: CC-07 idempotency.

Generated 2026-10-12 by scripts/update_todo_pdf.py